

COL783 Assignment-2

Sarthak Panda (2022CS51217), Ujjawal Sharma (2022CS51137)

PART - I

Question 1: Fourier Transform and Resizing

(a) Fourier Transform of a Scaled Image

An image can be modeled as a discrete sampling of an underlying continuous scene. Let this scene be $f_c(x, y)$, sampled with a uniform spacing of Δx and Δy . Scaling this image by a factor k is equivalent to resampling the same continuous scene with a new spacing, $\Delta x' = \Delta x/k$ and $\Delta y' = \Delta y/k$. Our goal is to find the relationship between the Fourier transforms of the original and scaled images.

The Convolution Theorem is fundamental to this analysis. It states that multiplication in the spatial domain corresponds to convolution in the frequency domain. Since a sampled image can be represented as the product of the continuous scene and a 2D impulse train (a Dirac comb), its Fourier transform will be the convolution of their respective transforms. This results in the key property of sampled signals: the Fourier transform of a sampled image is a periodic replication of the continuous scene's transform, with the replicas spaced at intervals related to the sampling rate.

(Original Sampled Signal)

$$f(x, y) = f_c(x, y) \cdot \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \delta(x - m\Delta x, y - n\Delta y)$$

(Fourier Transform of Original Signal)

$$F(\nu_u, \nu_v) = \frac{1}{\Delta x \Delta y} \sum_{p=-\infty}^{\infty} \sum_{q=-\infty}^{\infty} F_c \left(\nu_u - \frac{p}{\Delta x}, \nu_v - \frac{q}{\Delta y} \right)$$

(Fourier Transform of Scaled Signal)

$$F'(\nu_u, \nu_v) = \frac{1}{\Delta x' \Delta y'} \sum_{p=-\infty}^{\infty} \sum_{q=-\infty}^{\infty} F_c \left(\nu_u - \frac{p}{\Delta x'}, \nu_v - \frac{q}{\Delta y'} \right)$$

(Parameter Substitution for Scaling)

$$\frac{1}{\Delta x' \Delta y'} = \frac{1}{(\Delta x/k)(\Delta y/k)} = \frac{k^2}{\Delta x \Delta y}$$

$$\frac{p}{\Delta x'} = \frac{pk}{\Delta x} \quad \text{and} \quad \frac{q}{\Delta y'} = \frac{qk}{\Delta y}$$

(Final Relationship)

$$F'(\nu_u, \nu_v) = \frac{k^2}{\Delta x \Delta y} \sum_{p=-\infty}^{\infty} \sum_{q=-\infty}^{\infty} F_c \left(\nu_u - \frac{pk}{\Delta x}, \nu_v - \frac{qk}{\Delta y} \right)$$

Interpretation Comparing the transform of the scaled image, F' , with the original, F , reveals two primary effects of scaling:

1. **Amplitude Scaling:** The entire spectrum of the scaled image is multiplied by a factor of k^2 .

2. **Frequency Scaling:** The spacing between the periodic replications of the continuous spectrum F_c is scaled by a factor of k .

The implications depend on the value of k :

- **Zooming** ($k > 1$): The spectral copies move farther apart. This is a form of oversampling, which creates more "empty" space in the frequency domain between the spectral replicas.
- **Shrinking** ($0 < k < 1$): The spectral copies move closer together. If the original signal f_c is not sufficiently band-limited (i.e., if its transform F_c is too wide), the replicated copies will overlap as they are pushed together. This spectral overlap is known as **aliasing** and introduces artifacts into the shrunk image. This is why a low-pass filter must be applied before downsampling to prevent such overlap.

(b) Ideal Anti-Aliasing Filter

To prevent aliasing when shrinking an image (i.e., for $0 < k < 1$), the signal must be band-limited before resampling. The new, lower sampling rate cannot represent the high frequencies present in the original signal. An ideal **low-pass filter** is required to remove these high frequencies.

The filter's cutoff frequency must correspond to the Nyquist frequency of the *new* sampling grid. Since the new sampling interval is $\Delta x' = \Delta x/k$, the new sampling rate is $k/\Delta x$, and its Nyquist frequency is half of that rate. The ideal filter, often called a "brick-wall" filter, passes all frequencies below this new Nyquist limit and completely blocks all frequencies above it. In the frequency domain, this filter takes the shape of a 2D rectangular function.

By the Fourier transform's properties, a rectangular function in the frequency domain corresponds to a 2D **sinc** function in the spatial domain. To apply the filter, one would convolve this **sinc** kernel with the original image.

The ideal low-pass filter is a theoretical construct. The corresponding spatial **sinc** kernel extends infinitely and has negative lobes, which makes direct convolution computationally impractical and can introduce "ringing" artifacts near sharp edges. Therefore, real-world applications use finite approximations of the ideal filter, such as Gaussian filters or the kernels inherent in high-quality interpolation methods like bilinear or bicubic resampling.

(New Nyquist Cutoff Frequencies)

$$|\nu_u|_{\max} = \frac{k}{2\Delta x}, \quad |\nu_v|_{\max} = \frac{k}{2\Delta y}.$$

(Frequency-Domain Filter $H(\nu_u, \nu_v)$)

$$H(\nu_u, \nu_v) = \begin{cases} 1 & \text{if } |\nu_u| \leq \frac{k}{2\Delta x} \text{ and } |\nu_v| \leq \frac{k}{2\Delta y} \\ 0 & \text{otherwise} \end{cases}$$

or compactly, with $\text{rect}(z) = 1$ for $|z| \leq 1/2$:

$$H(\nu_u, \nu_v) = \text{rect}\left(\frac{\nu_u \Delta x}{k}\right) \text{rect}\left(\frac{\nu_v \Delta y}{k}\right)$$

(Spatial-Domain Kernel $h(x, y)$)

$$h(x, y) = \mathcal{F}^{-1}\{H(\nu_u, \nu_v)\}$$

$$h(x, y) = \frac{k^2}{\Delta x \Delta y} \text{sinc}\left(\frac{kx}{\Delta x}\right) \text{sinc}\left(\frac{ky}{\Delta y}\right)$$

$$\text{with the definition } \text{sinc}(z) = \frac{\sin(\pi z)}{\pi z}.$$

(c) Implementation and Results

Methodology Two variations of image resizing were implemented in Python to demonstrate the effects of aliasing. The standard 'barbara.bmp' test image was used due to its significant high-frequency detail in areas like the tablecloth and scarf.

1. **Naive Resizing:** This approach, implemented in the `resize_naive` function, uses a high-quality built-in interpolation method (`cv2.resize` with `INTER_CUBIC`). While effective for zooming, this method does not

apply an explicit anti-aliasing pre-filter tailored to the specific downsampling factor, k . It therefore serves as our control case, which is expected to produce aliasing artifacts when shrinking.

2. **Filtered Resizing:** This approach implements the correct anti-aliasing procedure derived in part (b). The `resize_filtered` function performs the following steps:

- The 2D Fast Fourier Transform (FFT) of the original image is computed.
- An ideal low-pass filter is created in the frequency domain. This filter is a rectangular mask of ones with a size of $(H \cdot k) \times (W \cdot k)$, where H and W are the original image dimensions. This size corresponds exactly to the new Nyquist frequency limits. All values outside this central mask are zero.
- The image’s spectrum is multiplied by this mask, effectively removing all high frequencies that cannot be represented at the new, lower resolution.
- The inverse FFT is applied to the filtered spectrum to return to the spatial domain.
- The resulting band-limited image is then downsampled by simply selecting every $1/k$ -th pixel, which is now safe to do without introducing aliasing.

For both methods, the shrunk images were zoomed back to their original dimensions using nearest-neighbor interpolation. This technique makes the pixel structure of the small images visible, allowing for a clear inspection of any aliasing artifacts. The experiment was run for shrink factors of $k = 1/2$, $k = 1/4$, and $k = 1/8$.

Results and Analysis The visual results clearly demonstrate the necessity of pre-filtering when downsampling an image.



Figure 1: Comparison of resizing methods for $k = 1/2$.



Figure 2: Comparison of resizing methods for $k = 1/4$.

In all cases, the **Naive Resizing** method introduces significant aliasing artifacts. These are most visible as Moiré patterns on the tablecloth and striped pants. Fine details are incorrectly sampled, creating false low-frequency



Figure 3: Comparison of resizing methods for $k = 1/8$.

patterns that were not present in the original image. As the shrink factor k decreases, the sampling rate becomes lower, and the aliasing artifacts become progressively more severe and visually distracting.

In contrast, the **Filtered Resizing** method produces a result that, while blurrier, is free from these false patterns. The blurriness is the correct and expected outcome of explicitly removing the high-frequency information that the target resolution cannot support. By sacrificing sharpness, we maintain the integrity of the image structure. This demonstrates that the frequency-domain filtering approach successfully prevents aliasing and produces a more faithful representation of the original image at a reduced size.

Question 2: Custom Blurring with Point Spread Functions

(a) Point Spread Function (PSF) Preparation

In an out-of-focus photograph, the **Point Spread Function (PSF)** represents the response of an imaging system to a point source of light. Typically, this function has a circular or disk-like shape corresponding to the physical aperture of the camera. However, by modifying the aperture geometry, one can obtain artistic blur effects with different PSF shapes (e.g., stars, polygons). In this task, we aim to construct and visualize PSFs of different scales that can later be used to simulate such unique blur effects on real images.

We created a star-shaped PSF pattern using a polygonal geometry defined by alternating outer and inner radii. The PSF was generated on a 200×200 canvas, ensuring a bright star shape on a dark background. Mathematically, the PSF can be expressed as:

$$h(x, y) \geq 0, \quad \text{and} \quad \sum_{x, y} h(x, y) = 1.$$

This normalization ensures that the total light energy remains constant when used for convolution-based blurring.

To generate PSFs of varying scales, we implemented a resizing function that uses bilinear interpolation to downsample the base PSF into smaller versions of sizes:

$$[150, 80, 40, 20, 10].$$

Each resized PSF was re-normalized so that its pixel intensities again summed to unity:

$$h_s(x, y) \leftarrow \frac{h_s(x, y)}{\sum_{x, y} h_s(x, y)}.$$

The PSF generation and resizing were implemented using Python and OpenCV. The following key functions were used:

- `make_star_psf()` — generates the initial star-shaped PSF with controllable parameters such as number of spikes and inner/outer radius.
- `resize_and_normalize_psf()` — resizes the PSF to multiple target sizes and normalizes each version.

Each PSF version was saved and visualized in a grid for comparison. Figure 4 shows the generated PSFs at different scales, demonstrating the loss of fine structure as the PSF size decreases.

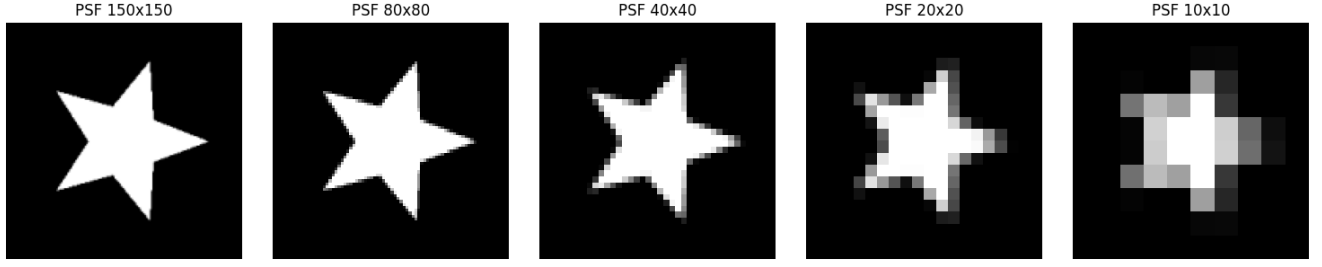


Figure 4: Star-shaped Point Spread Functions (PSFs) of varying sizes (150, 80, 40, 20, and 10 pixels), normalized to sum to unity.

Observations.

- As the PSF size decreases, the structural details (sharp spikes) of the star blur together, producing smoother kernels.
- Normalization ensures consistent brightness when used in convolution, independent of PSF size.
- The generated PSFs will be used in subsequent parts to simulate spatial and frequency domain blurs.

(b) Blurring in the Spatial Domain

The aim of this part is to perform image blurring by convolving an image with a Point Spread Function (PSF) in the **spatial domain**. The convolution should correctly handle kernel centering and image boundaries using *mirror padding*. The task requires demonstrating this blur on:

1. A synthetic image containing a few isolated impulses on a black background.
2. A real-world photograph of approximate size 1000×2000 pixels.

Blurring in the spatial domain is mathematically expressed as:

$$g(x, y) = (f * h)(x, y) = \sum_{u=-k}^k \sum_{v=-l}^l f(x-u, y-v) h(u, v),$$

where $f(x, y)$ is the input image, $h(u, v)$ is the PSF kernel, and $g(x, y)$ is the blurred output. To ensure that the PSF is properly centered, the kernel is flipped both horizontally and vertically before convolution. This was implemented using the function:

```
cv2.filter2D(img, -1, np.flipud(np.fliplr(psf)), borderType=cv2.BORDER_REFLECT)
```

Mirror (reflective) padding was used at the boundaries to prevent edge artifacts that occur with zero-padding.

The smallest PSF (of size 10×10) generated in Part (A) was used to perform the blurring for both the impulse image and the real photograph. All images were gamma-corrected before and after convolution to maintain correct perceptual brightness.

The following key functions were used:

- `spatial_blur_channel()` — Performs convolution on a single image channel with kernel flipping and reflective padding.
- `spatial_blur_color()` — Applies the above function across RGB channels with linear-domain gamma correction.

Results and Observations.

(i) Impulse Image. To visually verify the PSF behavior, a synthetic image containing three isolated impulses (bright pixels) was convolved with the star-shaped PSF. As shown in Figure 6, each impulse produces a blurred copy of the PSF centered at its position. This confirms that the convolution is working correctly and that the PSF is centered.

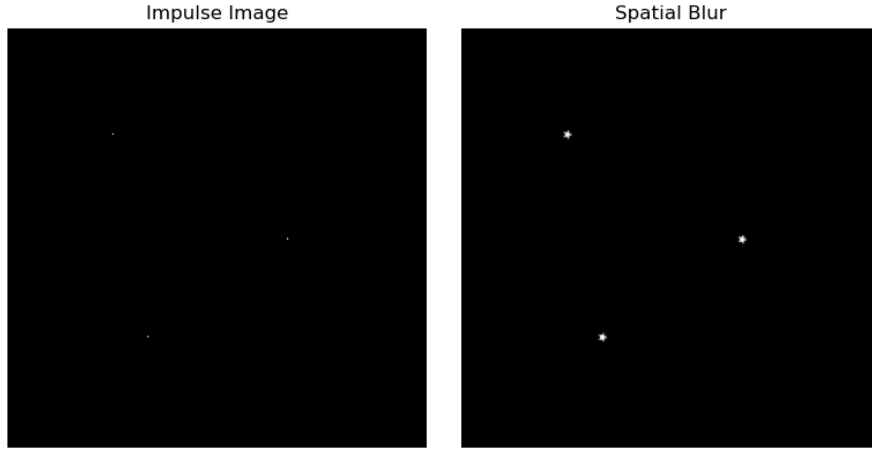


Figure 5: Impulse image (left) and its spatially blurred version (right) using a 10×10 star-shaped PSF. Each impulse spreads according to the PSF shape.

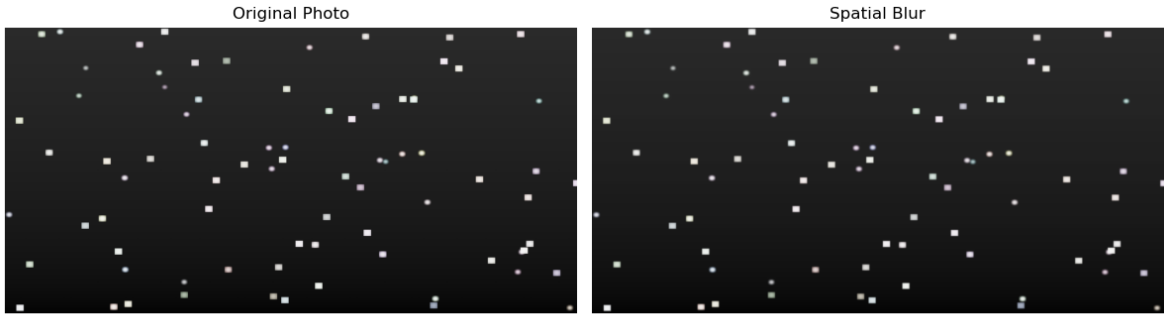


Figure 6: Impulse image (left) and its spatially blurred version (right) using a 10×10 star-shaped PSF. Each impulse spreads according to the PSF shape.

(ii) **Real Photograph.** The same convolution procedure was applied to a real-world photograph resized to approximately 1000×2000 pixels. The result, shown in Figure 7, demonstrates the spatial blur effect caused by the PSF. Fine details and edges in the image become softer, simulating an out-of-focus appearance consistent with a star-shaped aperture.



Figure 7: Original photograph (left) and its blurred version (right) after convolution with the smallest PSF. The result replicates an optical blur corresponding to a star-shaped aperture.

Discussion.

- The impulse test confirms that the spatial convolution process correctly aligns the PSF center with the impulse location.
- Mirror padding preserves edge continuity, avoiding dark border artifacts common in zero-padded convolutions.
- The real photograph exhibits natural blurring, with highlights diffused in a star-like pattern corresponding to the PSF structure.

(c) Blurring in the Frequency Domain

In this part, we implemented image blurring through convolution in the **frequency domain**. According to the Convolution Theorem, spatial-domain convolution between an image $f(x, y)$ and a kernel $h(x, y)$ is equivalent to point-wise multiplication in the frequency domain:

$$f(x, y) * h(x, y) \longleftrightarrow \mathcal{F}\{f(x, y)\} \cdot \mathcal{F}\{h(x, y)\}$$

where $\mathcal{F}\{\cdot\}$ denotes the Fourier Transform. The procedure followed was:

1. Mirror padding was applied to the image and kernel to avoid wrap-around artifacts during FFT.
2. Both the image and the kernel were transformed to the frequency domain using 2D FFT.
3. The complex-valued frequency spectra were multiplied element-wise.
4. The inverse FFT was taken to obtain the blurred image back in the spatial domain.
5. The resulting image was cropped to the original size.

This implementation ensures that the kernel h is properly centered before transformation, maintaining consistency with spatial convolution.

Results:

- The blurred outputs were visually indistinguishable from those obtained via spatial convolution.
- Quantitatively, the results matched up to floating-point rounding error.

Actual Photograph: Max abs diff: 0.01117, Mean abs diff: 0.00115
Synthetic Image: Max abs diff: 0.01943, Mean abs diff: 0.00081

These small numerical differences arise due to floating-point precision limits and boundary handling differences.

Displayed Results:

- **Impulse Image:** Comparison between original, spatially blurred, and frequency-domain blurred images.
- **Synthetic Image:** Intermediate visualizations of the padded image, magnitude spectra, and final blurred output (spatial vs. frequency).
- **Actual Photograph:** Intermediate FFT magnitude plots and final comparison between spatial-domain and frequency-domain blurring.

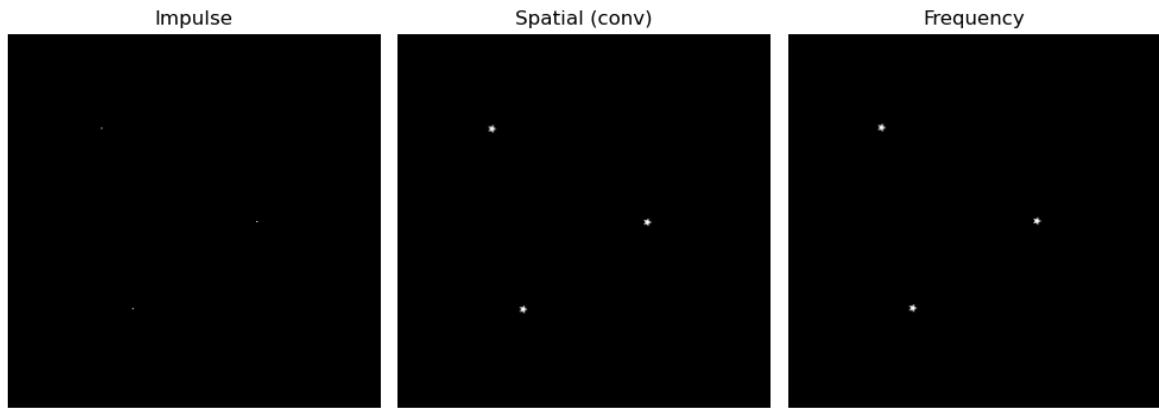


Figure 8: Impulse Image: Spatial vs. Frequency Domain Blurring

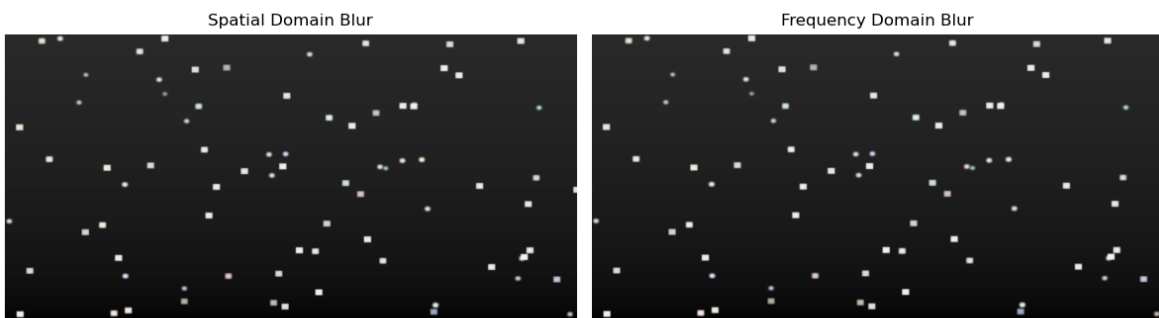


Figure 9: Synthetic Image: Spatial vs. Frequency Domain Blurring

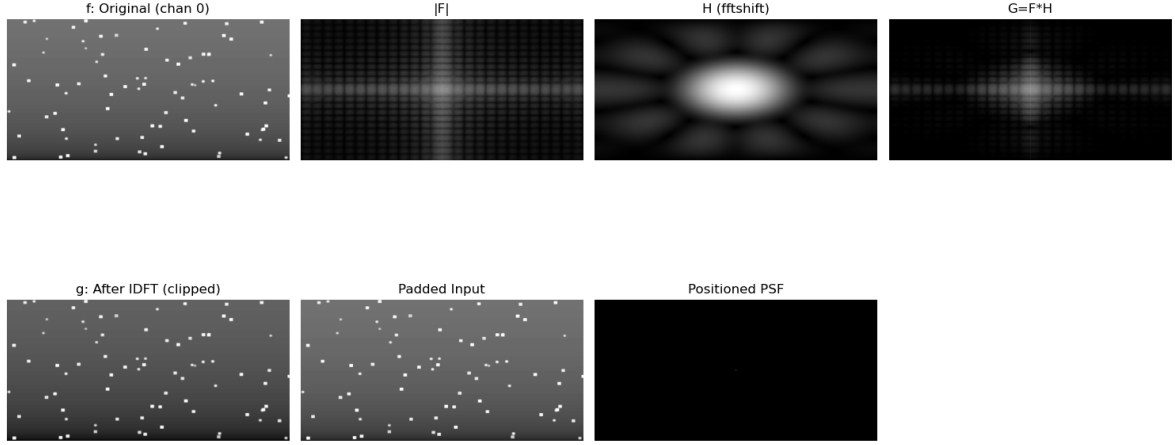


Figure 10: Synthetic Image: Frequency domain blurring with intermediate images



Figure 11: Actual Photograph: Spatial vs. Frequency Domain Blurring

Overall, the frequency-domain method produced results nearly identical to the spatial convolution method, confirming the correctness of both implementations.

(d) Performance Analysis

For each kernel size, denoted by $r = \max(m, n)$, the following procedure was performed:

1. Applied spatial-domain blurring using mirror padding and direct convolution.
2. Applied frequency-domain blurring using the FFT-based approach.
3. Recorded the wall-clock time for each implementation as $T_{\text{spatial}}(r)$ and $T_{\text{frequency}}(r)$.
4. Repeated the experiment on the same **actual photograph** for consistency.

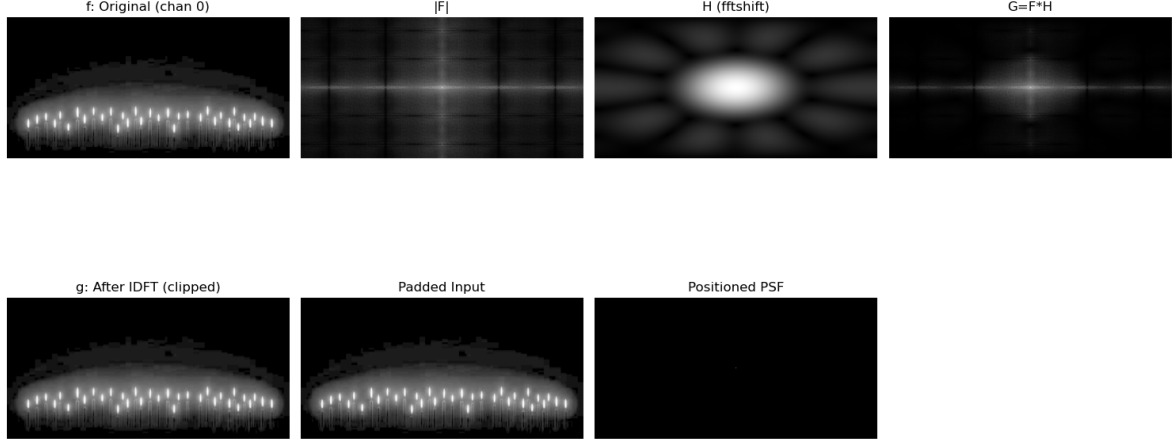


Figure 12: Actual Image: Frequency domain blurring with intermediate images

Results. The measured timings were plotted on a **log–log scale**, with the kernel size r on the x-axis and the computation time $T(r)$ on the y-axis. The resulting graph for the actual image is shown below.

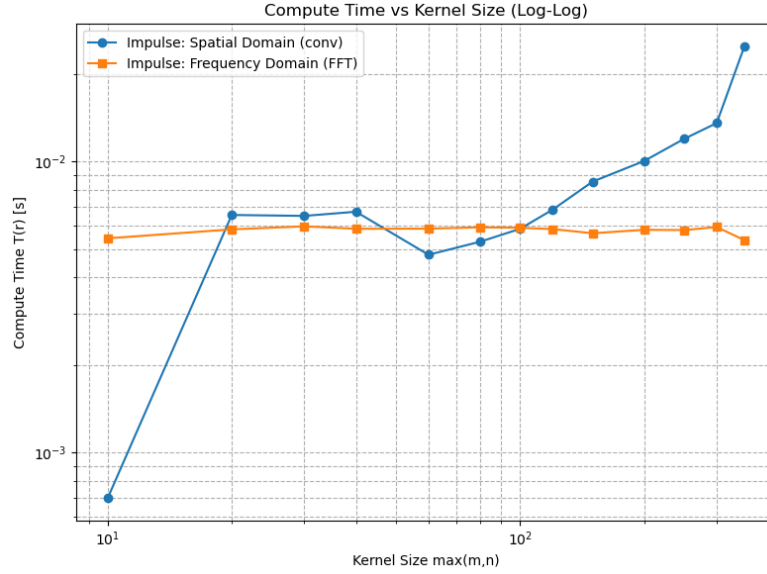


Figure 13: Computation time comparison for spatial and frequency domain blurring on the actual image.

Analysis. As observed from the log–log plot:

- For small kernels, the spatial convolution is faster due to its simpler implementation and lower constant factors.
- As the kernel size increases, the frequency-domain implementation becomes more efficient, reflecting its asymptotic advantage.

The crossover point, where the FFT-based method surpasses the spatial method, aligns with the expected theoretical behavior:

$$O(mn) \text{ (spatial)} \quad \text{vs.} \quad O(MN \log MN) \text{ (frequency-domain)}$$

for an $M \times N$ image.

Overall, the frequency-domain approach demonstrated superior scalability for large kernels, while spatial convolution remained preferable for small, localized filters.

Observation: Bokeh Effect

We observed the **bokeh effect** distinctly when using a large blur kernel of size 150×150 . The blurred regions exhibited the characteristic aperture-dependent shape of the point spread function (PSF), producing smooth circular highlights in out-of-focus areas.

Two representative examples are shown below:

1. **Synthetic Image:** Demonstrates how point sources expand according to the PSF shape.
2. **Actual Image:** Displays natural bokeh around bright regions and edges when convolved with the same PSF.



(a) Synthetic Image with 150×150 PSF



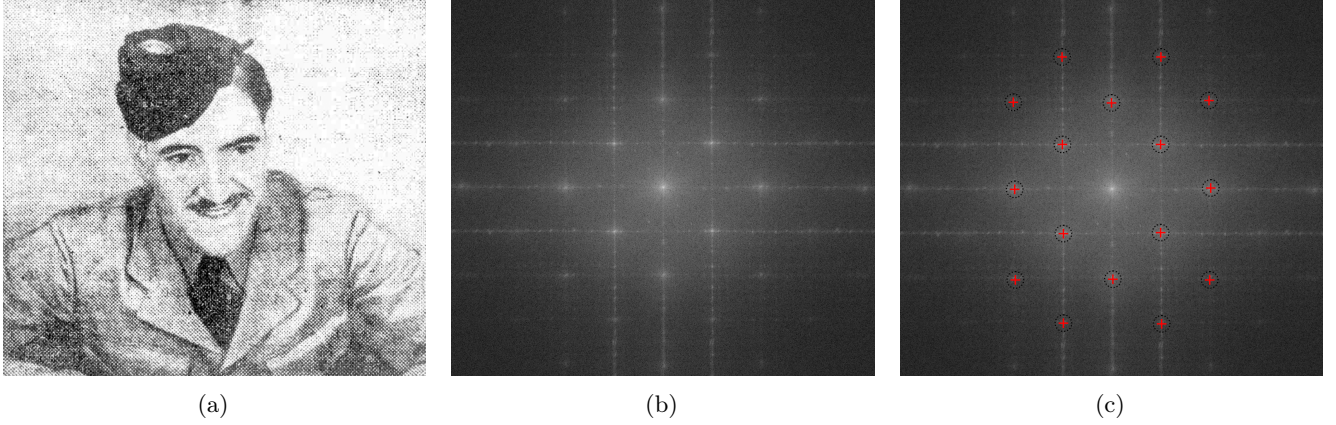
(b) Actual Image with 150×150 PSF

Figure 14: Observation of Bokeh Effect for large kernel size (150×150) in synthetic and real images.

————— **End of Part-I** —————

Q3: Halftone Dot Filtering

Part-A : Visualization of Fourier Spectrum



Fig(a) shows the Halftone Input Image, **Fig(b)** shows it's corresponding Fourier Spectrum with DC Component centered and In **Fig(c)** we have highlighted the Spikes corresponding to the halftone dot pattern

Let the image have height H and width W .

Define the frequency-domain center (using zero-based indexing) as $c_y = \left\lfloor \frac{H}{2} \right\rfloor$, $c_x = \left\lfloor \frac{W}{2} \right\rfloor$.

For a detected peak at pixel coordinates (r_y, r_x) , let

$$\Delta y = r_y - c_y, \quad \Delta x = r_x - c_x.$$

Normalized spatial frequencies (cycles per pixel) are

$$f_x = \frac{\Delta x}{W}, \quad f_y = \frac{\Delta y}{H}.$$

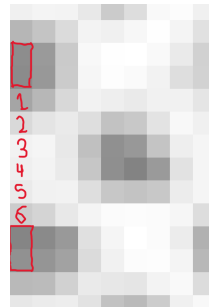
Radial (magnitude) frequency: $f = \sqrt{f_x^2 + f_y^2} = \sqrt{\left(\frac{\Delta x}{W}\right)^2 + \left(\frac{\Delta y}{H}\right)^2}$.

The spatial period (spacing) in *pixels per cycle* is the reciprocal of f :

$$\text{spacing}_{\text{px}} = \begin{cases} \frac{1}{f}, & f > 0, \\ \infty, & f = 0 \quad (\text{DC / zero-frequency — no periodicity}). \end{cases}$$

```
1: centroid=(351.0, 629.0), dy_dx=(-107.0, 117.0), fx_fy=(0.1142578125, -0.1168122270742358), f_mag=0.1634, spacing_px=6.1
2: centroid=(566.0, 395.0), dy_dx=(108.0, -117.0), fx_fy=(-0.1142578125, 0.11790393013100436), f_mag=0.1642, spacing_px=6.1
3: centroid=(351.0, 393.0), dy_dx=(-107.0, -119.0), fx_fy=(-0.1162109375, -0.1168122270742358), f_mag=0.1648, spacing_px=6.1
4: centroid=(563.0, 629.0), dy_dx=(105.0, 117.0), fx_fy=(0.1142578125, 0.1146288209606987), f_mag=0.1618, spacing_px=6.2
5: centroid=(456.0, 749.0), dy_dx=(-2.0, 237.0), fx_fy=(0.2314453125, -0.002183406113537118), f_mag=0.2315, spacing_px=4.3
6: centroid=(459.0, 278.0), dy_dx=(1.0, -234.0), fx_fy=(-0.228515625, 0.001091703056768559), f_mag=0.2285, spacing_px=4.4
7: centroid=(676.0, 514.0), dy_dx=(218.0, 2.0), fx_fy=(0.001953125, 0.23799126637554585), f_mag=0.2380, spacing_px=4.2
8: centroid=(251.0, 511.0), dy_dx=(-207.0, -1.0), fx_fy=(-0.0009765625, -0.2259825327510917), f_mag=0.2260, spacing_px=4.4
9: centroid=(245.0, 745.0), dy_dx=(-213.0, 233.0), fx_fy=(0.2275390625, -0.23253275109170304), f_mag=0.3253, spacing_px=3.1
10: centroid=(140.0, 392.0), dy_dx=(-318.0, -120.0), fx_fy=(-0.1171875, -0.3471615720524017), f_mag=0.3664, spacing_px=2.7
11: centroid=(249.0, 274.0), dy_dx=(-209.0, -238.0), fx_fy=(-0.232421875, -0.22816593886462883), f_mag=0.3257, spacing_px=3.1
```

(e)



(f)

Fig(e) shows the result of a python implementation to determine spacing between dots & **Fig(f)** shows zoomed version of input image for Verification of spacing between dots, The zoomed version is produced using VERIV VS Code extension

The above derivation explains the relation between the chosen spike coordinates (c_x, c_y) and the spacing between dots in pixels. An implementation of this logic in Python clearly shows that the dot spacing varies between 2 px and 6 px. The 6 px spacing corresponds to the lowest frequency (the four innermost points in **Fig. (c)** forming a square), which can be clearly observed near the boundary of the image when zoomed in.

Thanks to the VERIV VS Code extension, it was possible to directly count the pixels after zooming. This manual verification, along with the mathematically defined equations above, clearly establishes the relationship between frequency and dot spacing in the spatial domain.

Part-B : Notch Filtering using Gaussian & Butterworth Profile

Let us first look at the symmetric Notch filter equations for each of the filtering profile.

Given image/grid coordinates (y, x) , notch centers (r_k, s_k) , $k = 1, \dots, K$,

image center: (c_y, c_x) , and a small regularizer $\varepsilon > 0$.

Define the distance to the k -th center:

$$D_k(y, x) = \sqrt{(y - r_k)^2 + (x - s_k)^2} + \varepsilon.$$

The symmetric (conjugate) center is $(r'_k, s'_k) = (2c_y - r_k, 2c_x - s_k)$.

(1) Symmetric Gaussian notch-reject (per center)

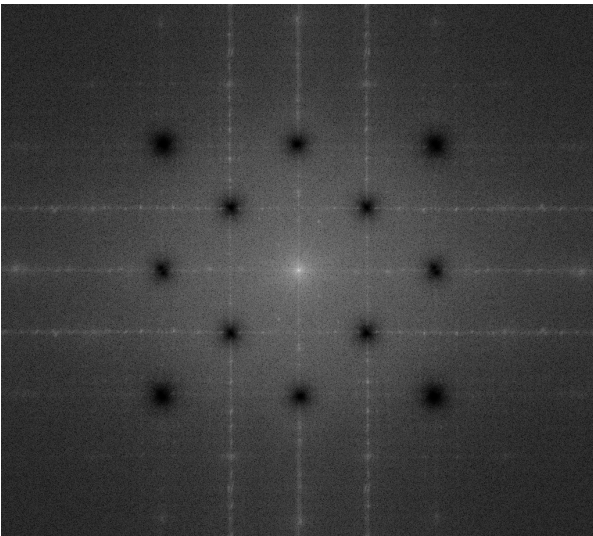
$$H_k^G(y, x) = 1 - \exp\left(-\frac{D_k(y, x)^2}{2 D_0^2}\right), \quad H_{k,\text{sym}}^G(y, x) = 1 - \exp\left(-\frac{D'_k(y, x)^2}{2 D_0^2}\right).$$

$$\text{Overall Gaussian mask: } H^G(y, x) = \prod_{k=1}^K H_k^G(y, x) H_{k,\text{sym}}^G(y, x).$$

(2) Symmetric Butterworth notch-reject (per center)

$$H_k^B(y, x) = \frac{1}{1 + \left(\frac{D_0}{D_k(y, x)}\right)^{2n}}, \quad H_{k,\text{sym}}^B(y, x) = \frac{1}{1 + \left(\frac{D_0}{D'_k(y, x)}\right)^{2n}}.$$

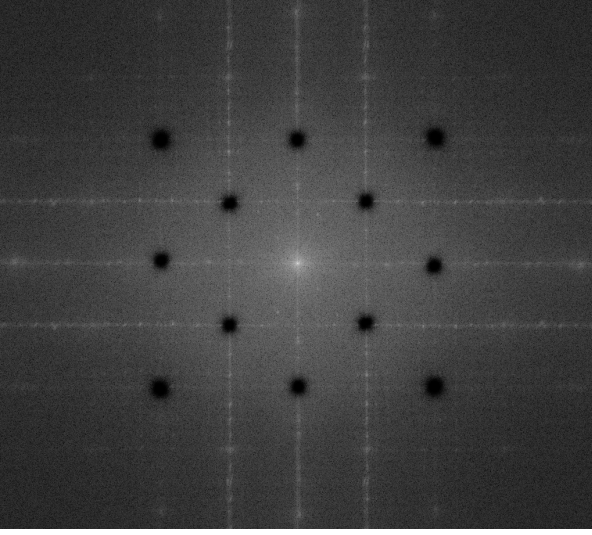
$$\text{Overall Butterworth mask: } H^B(y, x) = \prod_{k=1}^K H_k^B(y, x) H_{k,\text{sym}}^B(y, x).$$



Filtered Frequency Spectrum for Gaussian Profile



Resulting Filtered Image [Gaussian Profile]



Filtered Frequency Spectrum for Butterworth Profile



Resulting Filtered Image [Butterworth Profile]

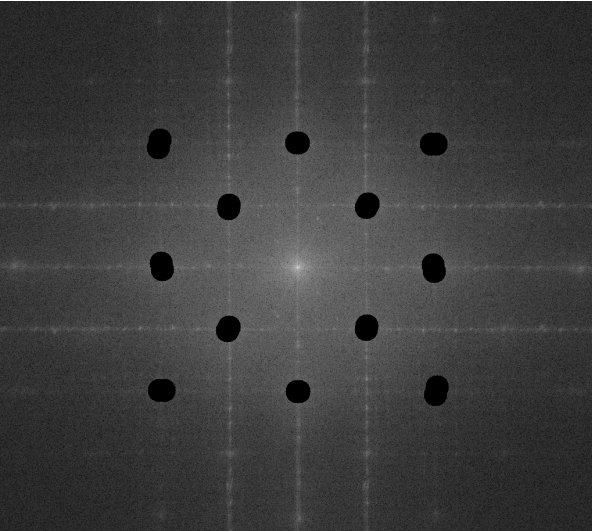
The above images show the filtered frequency spectrum and the resulting images obtained from both the Symmetric Gaussian Notch Filter (with $\sigma = 20$) and the Symmetric Butterworth Notch Filter (with $n = 2$ and $D_0 = 20$). In both methods, if we carefully observe the black hat and collar regions of the photograph, we can see that the unwanted dot patterns have been removed. Hence, we can conclude that the implemented notch filtering works as expected.

Part-C : Notch Filtering using Ideal filter

Symmetric Ideal notch-reject (per center)

$$H_k^I(y, x) = \begin{cases} 0, & D_k(y, x) \leq R, \\ 1, & D_k(y, x) > R, \end{cases} \quad H_{k,\text{sym}}^I(y, x) = \begin{cases} 0, & D'_k(y, x) \leq R, \\ 1, & D'_k(y, x) > R, \end{cases}$$

$$\text{Overall Ideal mask: } H^I(y, x) = \prod_{k=1}^K H_k^I(y, x) H_{k,\text{sym}}^I(y, x).$$



Filtered Frequency Spectrum for Ideal Notch Filter



Resulting Filtered Image [Ideal Notch Filter]

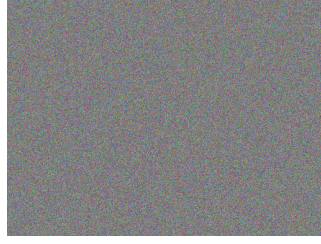
The results in this section are very similar to those observed in the previous section. The only additional effect we expected here was ringing (Gibbs-like oscillations) from the ideal (hard-cutoff) notch, but the ringing is not clearly visible in the filtered image. This is most likely because the spectral dots we removed contain very little energy, when the removed energy is small the amplitude of any resulting ringing is also small and it is hard to see them.

Q4: Denoising with Mean, Median & NLM Filtering [*Optional Solved*]

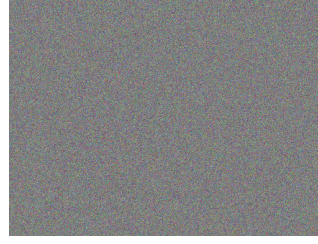
Part-A : Synthetic Noisy Image Generation



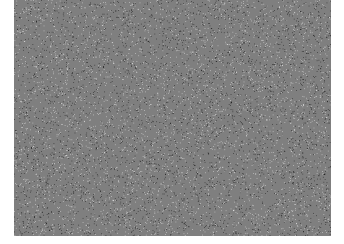
(a) Constant Intensity Image



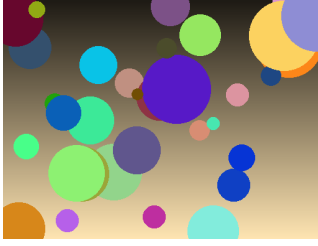
(b) With Uniform Noise



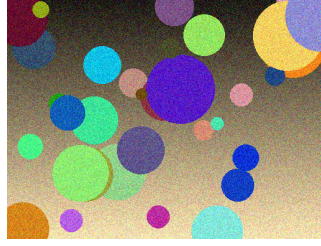
(c) With Gaussian Noise



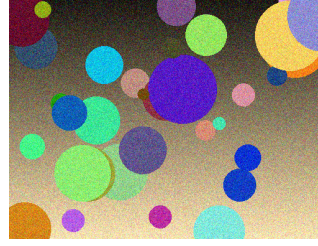
(d) With Salt-Pepper Noise



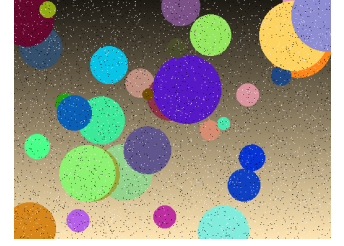
(a) Natural Image



(b) With Uniform Noise



(c) With Gaussian Noise



(d) With Salt-Pepper Noise

The above images show the original images and the results after adding synthetic noise. The main step in generating this noise is converting a target PSNR value to a mean squared error (MSE), and then choosing a noise of required type whose variance equals that MSE. Note that this result holds only for zero mean Noise. Below is a short derivation of this result.

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right) \Rightarrow \text{MSE}_{\text{required}} = \frac{\text{MAX}_I^2}{10^{\text{PSNR}_{\text{target}}/10}}$$

Where the Mean Squared Error is defined as: $\text{MSE} = \frac{1}{MN} \sum_{i,j} [I(i,j) - K(i,j)]^2$

Given Additive Noise: $K = I + \eta \Rightarrow [I - K]^2 = \eta^2$

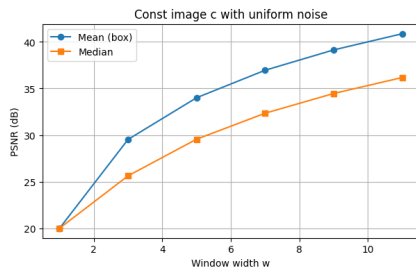
Thus, $\text{MSE} = \frac{1}{MN} \sum \eta^2 = \text{E}[\eta^2]$

Noise Variance: $\sigma_{\text{noise}}^2 = \text{E}[\eta^2] - (\text{E}[\eta])^2$

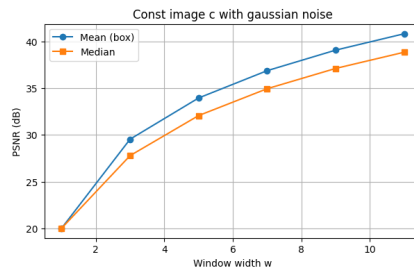
For Zero-Mean Noise ($\text{E}[\eta] = 0$): $\sigma_{\text{noise}}^2 = \text{E}[\eta^2]$

Therefore, $\sigma_{\text{required}}^2 = \text{MSE}_{\text{required}}$

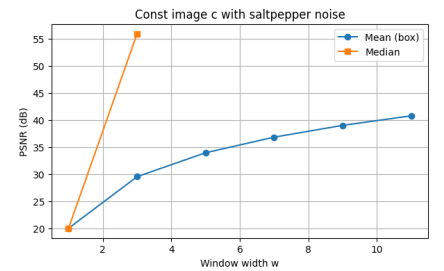
Part-B : Denoising Constant Intensity Image with Mean & Median Filter



(a) PSNR v/s W for Uniform Noise



(b) PSNR v/s W for Gaussian Noise

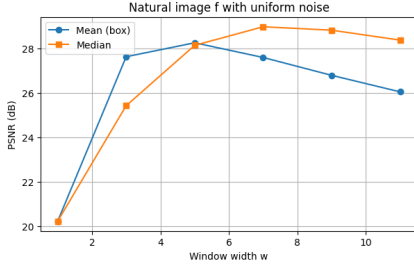


(c) PSNR v/s W for Salt-Pepper Noise

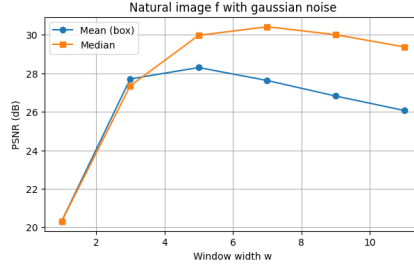
There are a few important observations from the above plots. The main observation is that for a constant-intensity image, irrespective of noise type or denoising method (i.e., mean or median filtering), increasing the window size improves PSNR. In other words, larger windows produce stronger denoising. This is because for zero-mean noise, averaging over a larger region tends to cancel the noise and restore the original pixel intensities; a similar intuition applies to the median filter.

A second observation is that the mean filter is more effective for uniform noise, whereas the median filter outperforms the mean filter for salt-and-pepper noise. In our experiments the median filter completely removed salt-and-pepper noise for window sizes 5×5 and larger; in those cases the measured PSNR values become infinite (∞) or undefined (NaN), so they are omitted from the plot.

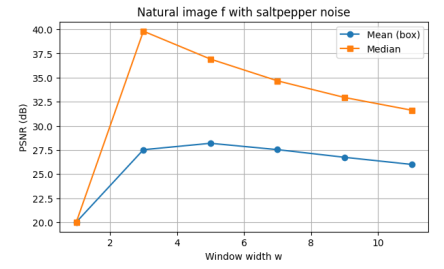
Part-C : Denoising Natural Image with Mean & Median Filter



(a) PSNR v/s W for Uniform Noise



(b) PSNR v/s W for Gaussian Noise



(c) PSNR v/s W for Salt-Pepper Noise

The observations here are opposite to those in the previous section. As noted in the problem, PSNR increases with the window width up to a certain point, and then decreases. Thus, increasing the window width to a very large value does not improve denoising. This is expected because natural images do not exhibit constant intensity across large regions for example averaging over the entire image tends to produce a nearly constant image that differs substantially from the original. An intermediate (optimum) window size performs best since the intensity can be assumed approximately constant only within a small neighborhood around a pixel.

Consistent with previous results, the median filter clearly outperforms the mean filter for salt-and-pepper noise. For this input image the maximal PSNR obtained with the median filter curves is higher than the maximal PSNR obtained with the mean filter for each noise type tested.

Part-D : Denoising with NLM

$$\hat{u}(i) = \frac{1}{Z(i)} \sum_{j \in \Omega} w(i, j) v(j), \quad Z(i) = \sum_{j \in \Omega} w(i, j),$$

$$w(i, j) = \exp\left(-\frac{\tilde{d}(i, j)}{h^2}\right),$$

$$\tilde{d}(i, j) = \max\left(\|v(N_i) - v(N_j)\|_{2,G}^2 - 2\sigma^2 |N|, 0\right),$$

$$\|v(N_i) - v(N_j)\|_{2,G}^2 = \sum_{k \in N} G(k) (v(i+k) - v(j+k))^2,$$

$$G(k) = \exp\left(-\frac{\|k\|^2}{2\sigma_G^2}\right), \quad \sigma_G = \frac{\text{patch_radius}}{2},$$

$$(\text{we normalize the Gaussian kernel so that } \sum_{k \in N} G(k) = 1).$$

Notation: $j \in \Omega$ denotes the *search window* (pixels searched for similarity),
 $k \in N$ denotes the *patch window* (patch pixels indexed by k).

$$(\text{Noise bias identity}) \quad \mathbb{E}[\|v(N_i) - v(N_j)\|_2^2] = \|u(N_i) - u(N_j)\|_2^2 + 2\sigma^2 |N|.$$

Note: σ and h are algorithm parameters (σ can be estimated from the image). In practice $h \approx 0.8 \sigma$ is commonly used for a faster/conservative setting, while $h \approx 1.0 \sigma$ is used for higher-quality denoising.

Clarifications: $v(i + k)$ denotes the intensity of the k -th pixel in the patch (neighbourhood) centred at i , and $v(j + k)$ is the intensity of the corresponding k -th pixel in the patch centred at j .

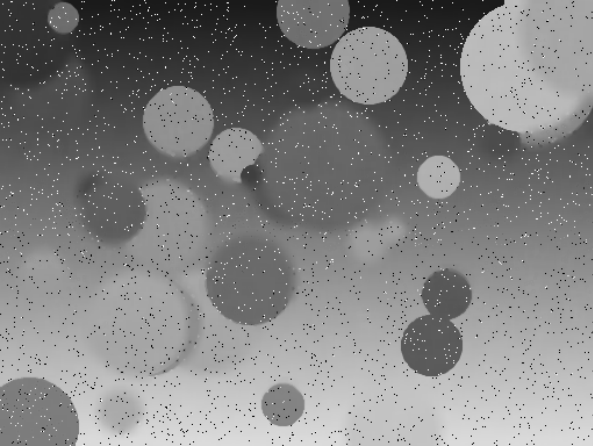
$\|k\|$ denotes the spatial (Euclidean) distance from the patch center to the k -th neighbour in the patch window.



(a) NLM on Uniform Noise



(b) NLM on Gaussian Noise



(c) NLM on Salt-Pepper Noise



(d) Adaptive Median + NLM on S&P noise

PSNR Values across different denoising strategies				
Noise Type	Mean	Median	NLM	Adaptive Median + NLM
Uniform	28.267dB	28.988dB	31.787dB	N/A
Gaussian	28.298dB	30.415dB	31.947dB	N/A
Salt-and-Pepper	28.191dB	39.810dB	21.188dB	35.453dB

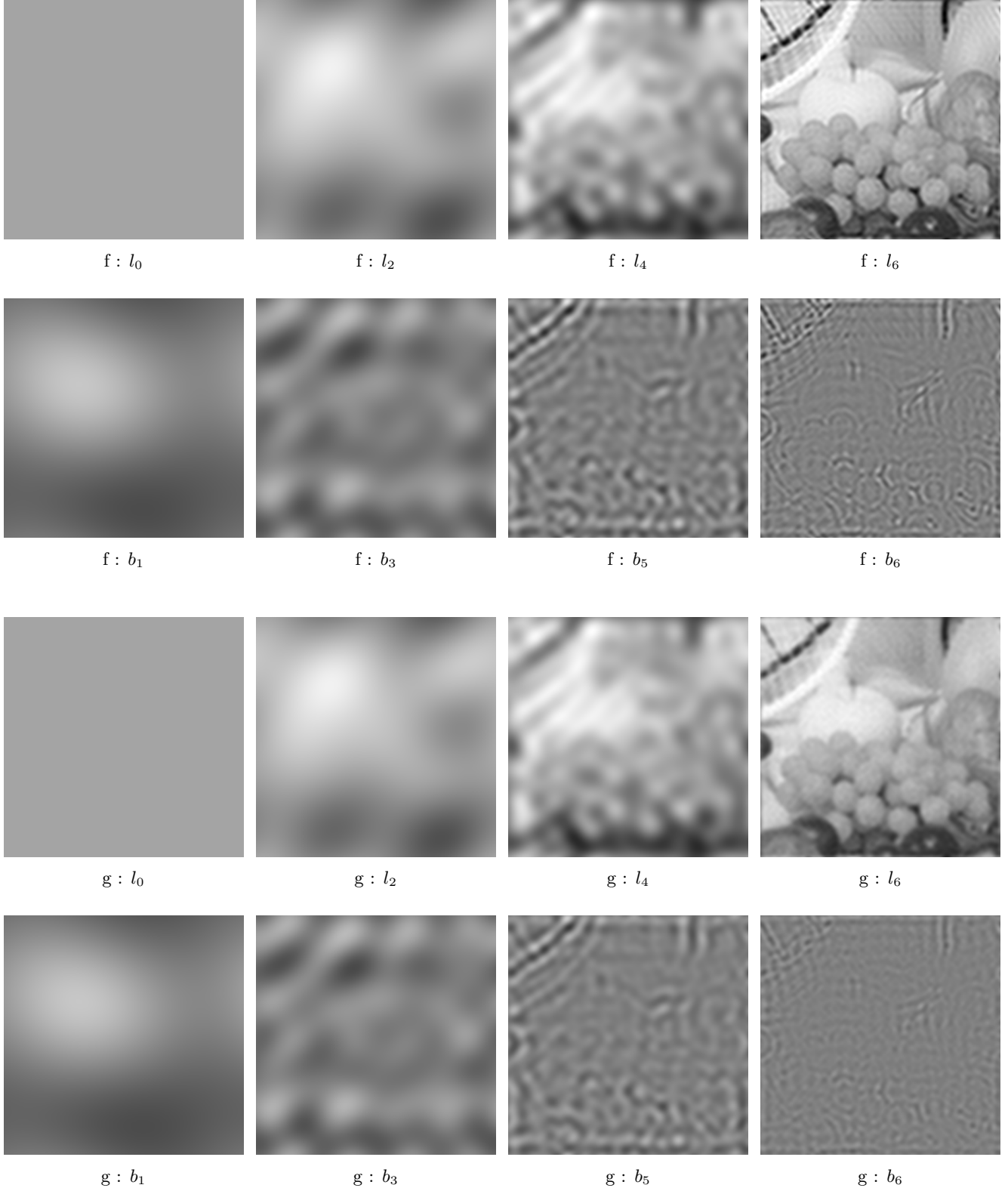
The above images show the results obtained using Non-Local Means (NLM) filtering. For this experiment, we used a **patch radius of 2** and a **search radius of 6**. Each run took approximately **500–600 seconds**, which is expected since our implementation is a naive brute-force version and the image size was around 384×512 . There may exist better parameter settings that can yield even higher-quality results; however, with the current configuration, the PSNR values obtained from NLM are summarized in the above table. For Mean and Median filtering, we used the best results corresponding to the peak points of their respective PSNR curves from the previous section.

It is evident that for **Uniform** and **Gaussian** noise, the NLM filter performs better than both Mean and Median filters. However, one significant limitation of NLM is its poor performance on **Salt & Pepper** noise. To address

this, we first applied an *adaptive median filter* to remove most of the impulse noise, and then performed NLM filtering on the preprocessed image. This hybrid approach produced better results compared to direct NLM filtering. Visually, the noise artifacts are significantly reduced for Uniform and Gaussian noise, and the images retain sharper and cleaner edges, especially near the top corners. The only remaining issue is blurriness in some regions, which can likely be corrected using sharpening filters or other post-processing techniques.

Q5: Deblurring & Denoising with Wiener Filtering [*Optional Solved*]

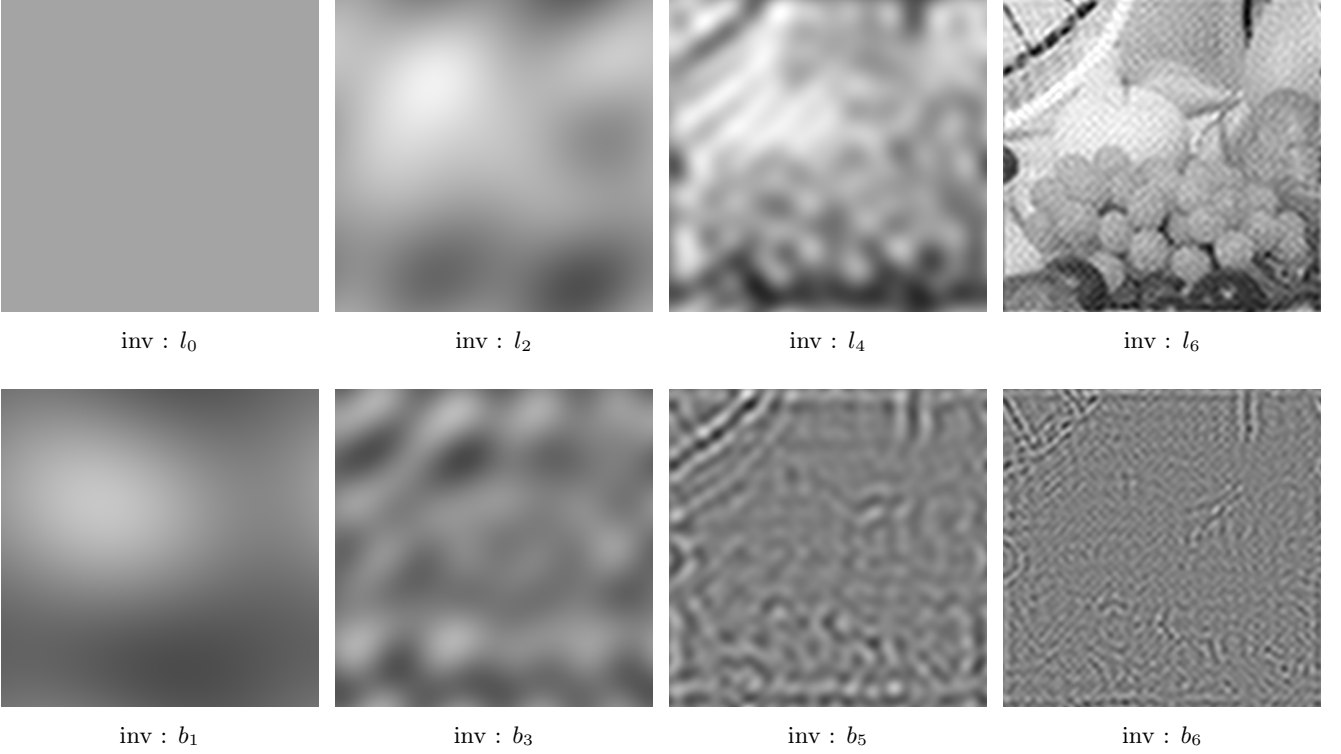
Part-A : Visualizing low-pass and band-pass filtered images



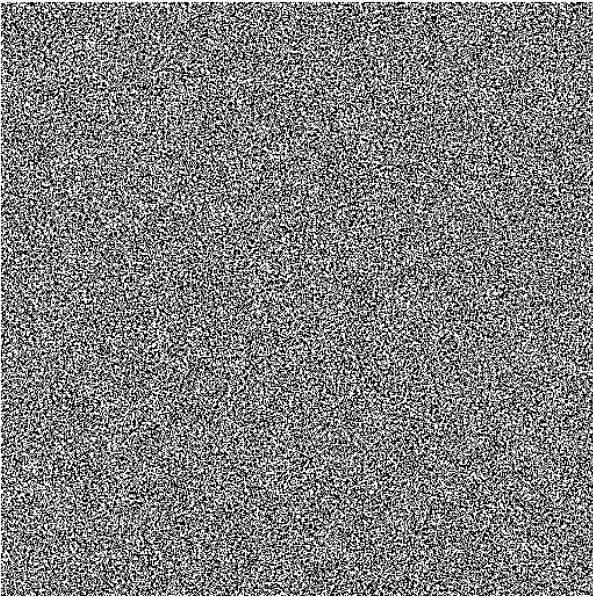
$$\begin{aligned} \mathbf{g} &= \mathbf{h} * \mathbf{f} + \boldsymbol{\eta} \\ \mathbf{G} &= \mathbf{H}\mathbf{F} + \mathbf{N} \end{aligned}$$

The above images show the low-pass and band-pass filtered versions of the input images f and g . Here, h denotes the known blur function, for which a Gaussian PSF has been used.

Part-B : Inverse Filtered Image & It's Frequency Bands



$$\hat{F} = \frac{G}{H} = \frac{FH + N}{H} = F + \frac{N}{H}$$



Inverse filtered Image

If we observe carefully, the inverse-filtered image appears as a noisy haze. Its PSNR relative to the original image is negative (around -36.224 dB), which indicates that $\|f - \hat{f}\|^2$ is much larger than $I_{\max}^2$. This was expected, given how dissimilar the restored image looks.

In contrast, the low-pass and band-pass filtered images still retain some visible information about the original image. This difference in observation can be easily explained by the noise amplification caused by the $\frac{N}{H}$ term in naive inverse filtering, which results in poor restoration and makes the final output appear highly dissimilar to the original image.

Part-C : Estimating Wiener Filter Parameters

Derivation of Constant Wiener Filter Ratio K_{est}

Assumptions: Image $M \times N$, Noise Variance σ^2 , Constant PSDs. $K_{\text{est}} = \frac{S_n}{S_f}$

1. Estimate Noise PSD $S_n = E[|N|^2]$:

Spatial Energy (zero mean noise): $\sum_{x,y} |\eta|^2 \approx MN\sigma^2$
 Frequency Energy (by Parseval): $\frac{1}{MN} \sum_{u,v} |N|^2 = \sum_{x,y} |\eta|^2$
 Therefore: $S_n = E[|N|^2] = MN\sigma^2$

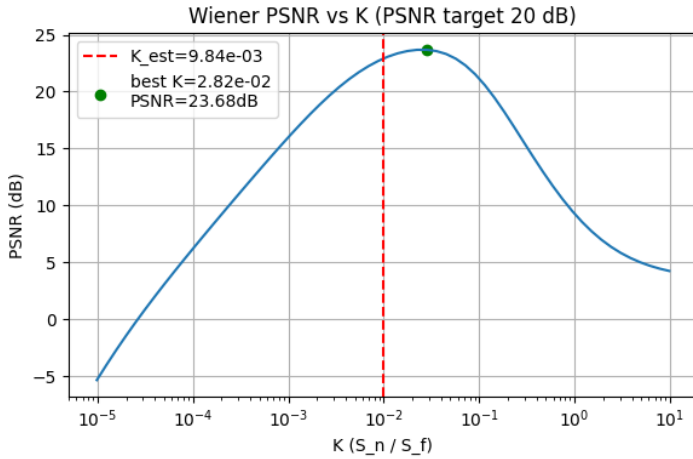
2. Estimate Image PSD $S_f = E[|F|^2]$:

Spatial Energy (Squared Norm): $\sum_{x,y} |f|^2 = \|f\|_2^2$
 Frequency Energy (by Parseval): $\frac{1}{MN} \sum_{u,v} |F|^2 = \sum_{x,y} |f|^2$
 Therefore: $S_f = E[|F|^2] = \|f\|_2^2$

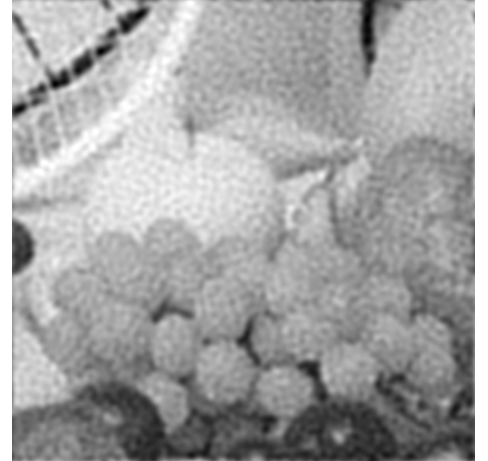
3. Final Ratio K_{est} :

For artificially added noise: $\sigma = \sigma_{20dB} = \sqrt{\frac{\text{mean}((h*f)^2)}{10^{(\text{target_psnr_db}/10)}}$
 Final ratio: $K_{\text{est}} = \frac{S_n}{S_f} = \frac{MN\sigma^2}{\|f\|_2^2}$

Part-D : Optimizing Wiener Filter Parameters for 20dB Noise Reduction



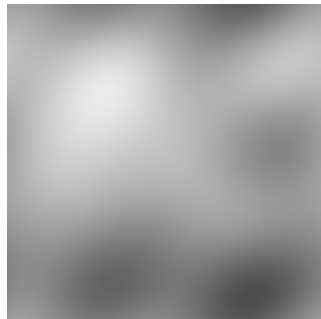
PSNR v/s K(Wiener param.) plot



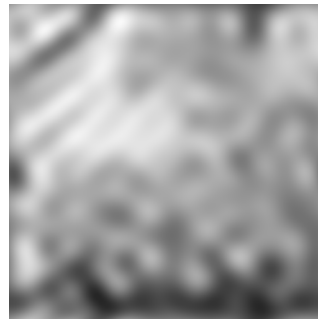
Best Restoration from Wiener Filter



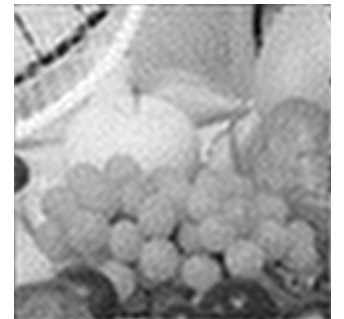
wien : l_0



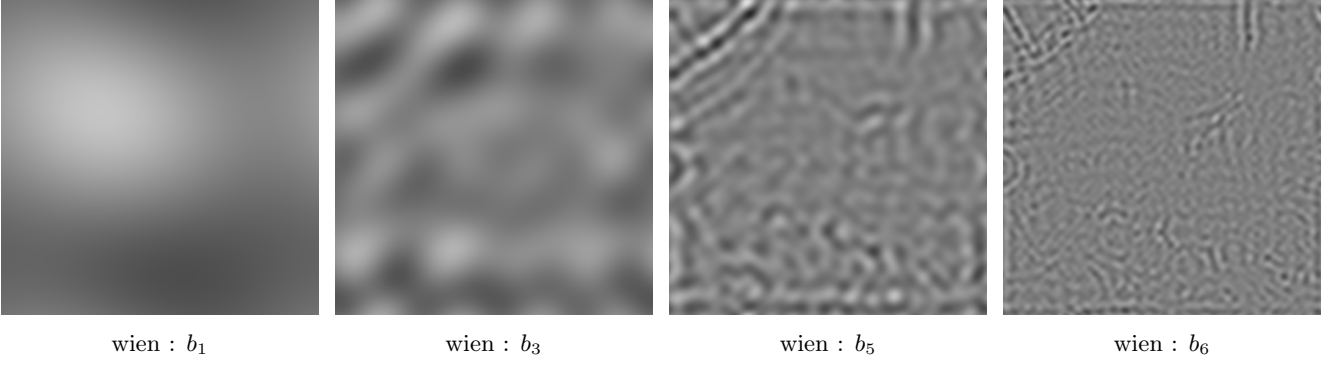
wien : l_2



wien : l_4



wien : l_6



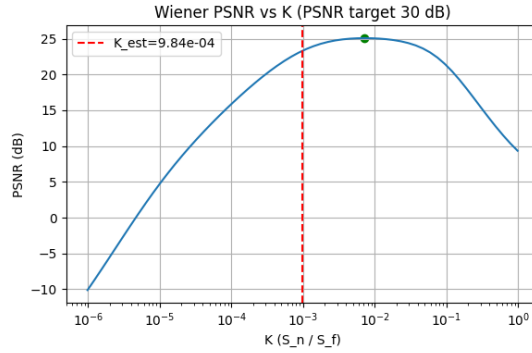
$$\hat{\mathbf{F}}(u, v) = \left[\frac{\mathbf{H}^*(u, v)}{|\mathbf{H}(u, v)|^2 + K} \right] \mathbf{G}(u, v)$$

The best restoration obtained by varying the Wiener filter parameter was around 23.679 dB, which is significantly better compared to direct inverse filtering. However, considering that we started with a noisy and blurred image having a noise level of 20 dB, this improvement is not very substantial. Also note that K estimated is close to best K parameter.

Part-E : Optimizing Wiener Filter Parameters for 10dB & 30dB Noise Reduction



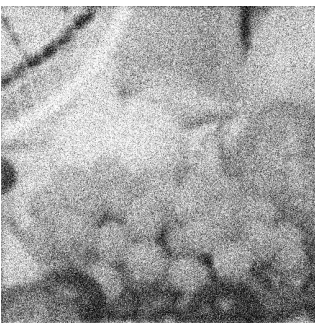
Input Image (30dB Noise)



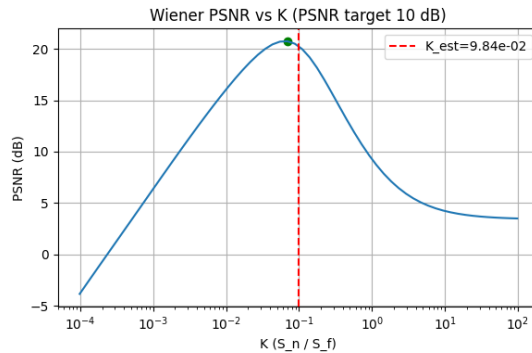
PSNR v/s K (Wiener param.) plot



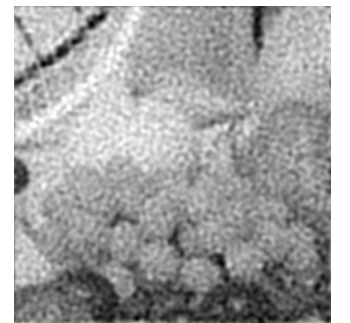
Best Restoration



Input Image (10dB Noise)



PSNR v/s K (Wiener param.) plot



Best Restoration

If we examine the results, we observe the following. Starting from a less noisy input (about 30 dB), the best PSNR obtained with the Wiener filter was roughly 25 dB (i.e., slightly worse than the input PSNR), although the image appearance improved somewhat because the blur was reduced. For a much noisier input (about 10 dB), the best Wiener restoration reached approximately 21 dB. Thus, the Wiener filter produces a more pronounced noise reduction when the input is heavily noisy, while the de-blurring effect is more noticeable when the input has relatively low noise. This demonstrates wiener filter behavior is sensitive to signal to noise ratio, where in high signal to noise ratio wiener effectively becomes inverse filter and does more effective de-blurring, while low signal to noise ratio makes wiener filter's behavior opposite.

Part-F : Regularized De-convolution

Let us first find a closed form expression for optimal $\hat{\mathbf{f}}$ with the give Regularizer, Then we will look at the results from this de-convolution. The objective is to find the estimated image $\hat{\mathbf{f}}$ that minimizes the cost function $E(\hat{\mathbf{f}})$:

$$\hat{\mathbf{f}}_{\text{opt}} = \arg \min_{\hat{\mathbf{f}}} \left\{ E(\hat{\mathbf{f}}) \right\}$$

where $E(\hat{\mathbf{f}})$ combines the Data Fidelity Term and the Regularization Term:

$$E(\hat{\mathbf{f}}) = \underbrace{\|\mathbf{g} - \mathbf{h} * \hat{\mathbf{f}}\|_2^2}_{\text{Data Fidelity}} + \lambda \underbrace{R(\hat{\mathbf{f}})}_{\text{Regularization}}$$

The regularization term $R(\hat{\mathbf{f}})$ is the squared gradient norm:

$$R(\hat{\mathbf{f}}) = \sum_{x,y} \left((\hat{f}_{x+1,y} - \hat{f}_{x,y})^2 + (\hat{f}_{x,y+1} - \hat{f}_{x,y})^2 \right) = \|\mathbf{D}_x \hat{\mathbf{f}}\|_2^2 + \|\mathbf{D}_y \hat{\mathbf{f}}\|_2^2$$

where $\mathbf{D}_x$ and $\mathbf{D}_y$ are the forward difference operators.

Step 1: Convert to the Frequency Domain using Parseval's Theorem

We use Parseval's Theorem ($\|\mathbf{x}\|_2^2 = \frac{1}{MN} \|\mathbf{X}\|_2^2$) and the Convolution Theorem ($\mathcal{F}\{\mathbf{h} * \hat{\mathbf{f}}\} = \mathbf{H}\hat{\mathbf{F}}$) to express $E(\hat{\mathbf{f}})$ in terms of Fourier Transforms (FTs).

A. Data Fidelity Term

$$\|\mathbf{g} - \mathbf{h} * \hat{\mathbf{f}}\|_2^2 = \frac{1}{MN} \sum_{u,v} |\mathbf{G}(u,v) - \mathbf{H}(u,v)\hat{\mathbf{F}}(u,v)|^2$$

B. Regularization Term

The FT of the 1D forward difference operator is $\mathbf{D}_x(u) = 1 - e^{-j2\pi u/M}$. Its squared magnitude is:

$$|\mathbf{D}_x(u)|^2 = |1 - e^{-j2\pi u/M}|^2 = 2 - 2 \cos\left(\frac{2\pi u}{M}\right) = 4 \sin^2\left(\frac{\pi u}{M}\right)$$

Similarly, $|\mathbf{D}_y(v)|^2 = 4 \sin^2\left(\frac{\pi v}{N}\right)$.

The regularization term in the FT domain is:

$$R(\hat{\mathbf{f}}) = \frac{1}{MN} \sum_{u,v} |\hat{\mathbf{F}}(u,v)|^2 \underbrace{(|\mathbf{D}_x(u,v)|^2 + |\mathbf{D}_y(u,v)|^2)}_{\mathbf{P}(u,v)}$$

where $\mathbf{P}(u,v)$ is the energy of the gradient operator in the frequency domain:

$$\mathbf{P}(u,v) = 4 \left(\sin^2\left(\frac{\pi u}{M}\right) + \sin^2\left(\frac{\pi v}{N}\right) \right)$$

C. Full Objective Function in FT

Dropping the constant factor $\frac{1}{MN}$, the objective is to minimize the sum, which means minimizing the term inside the summation for each frequency component (u,v) . We focus on the scalar objective function $E_{u,v}(\hat{F})$:

$$E_{u,v}(\hat{F}) = |\mathbf{G} - \mathbf{H}\hat{F}|^2 + \lambda \mathbf{P}|\hat{F}|^2$$

Step 2: Minimize $E_{u,v}(\hat{F})$ using the Wirtinger Derivative

Expand the squared magnitudes using $|\mathbf{Z}|^2 = \mathbf{Z}\mathbf{Z}^*$:

$$\begin{aligned} E_{u,v} &= (\mathbf{G} - \mathbf{H}\hat{F})(\mathbf{G}^* - \mathbf{H}^*\hat{F}^*) + \lambda \mathbf{P}\hat{F}\hat{F}^* \\ E_{u,v} &= |\mathbf{G}|^2 - \mathbf{G}\mathbf{H}^*\hat{F}^* - \mathbf{G}^*\mathbf{H}\hat{F} + |\mathbf{H}|^2|\hat{F}|^2 + \lambda \mathbf{P}|\hat{F}|^2 \\ E_{u,v} &= |\mathbf{G}|^2 - \mathbf{G}\mathbf{H}^*\hat{F}^* - \mathbf{G}^*\mathbf{H}\hat{F} + (|\mathbf{H}|^2 + \lambda \mathbf{P})|\hat{F}|^2 \end{aligned}$$

Set the derivative with respect to $\hat{F}^*$ to zero to find the minimum:

$$\frac{\partial E_{u,v}}{\partial \hat{F}^*} = 0$$

The derivatives are:

$$\begin{aligned}\frac{\partial}{\partial \hat{F}^*} (|\mathbf{G}|^2) &= 0 \\ \frac{\partial}{\partial \hat{F}^*} (-\mathbf{G}\mathbf{H}^* \hat{F}^*) &= -\mathbf{G}\mathbf{H}^* \\ \frac{\partial}{\partial \hat{F}^*} (-\mathbf{G}^* \mathbf{H} \hat{F}) &= 0 \\ \frac{\partial}{\partial \hat{F}^*} \left((|\mathbf{H}|^2 + \lambda \mathbf{P}) \hat{F} \hat{F}^* \right) &= (|\mathbf{H}|^2 + \lambda \mathbf{P}) \hat{F}\end{aligned}$$

Substituting back:

$$-\mathbf{G}\mathbf{H}^* + (|\mathbf{H}|^2 + \lambda \mathbf{P}) \hat{\mathbf{F}} = 0$$

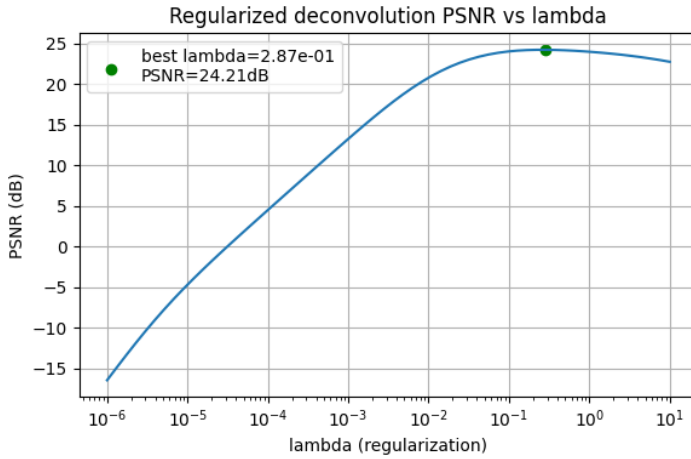
Step 3: Closed-Form Solution

Solving for $\hat{\mathbf{F}}$:

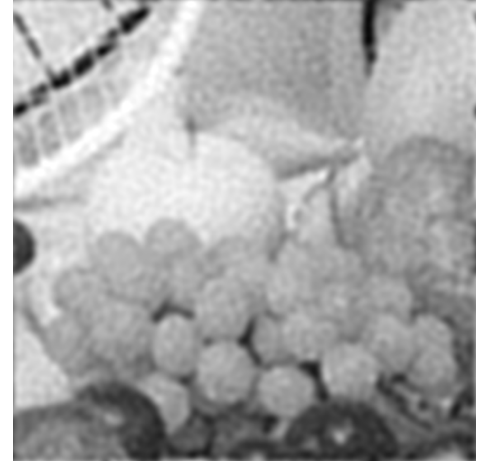
$$\hat{\mathbf{F}}(u, v) = \frac{\mathbf{G}(u, v) \mathbf{H}^*(u, v)}{|\mathbf{H}(u, v)|^2 + \lambda \mathbf{P}(u, v)}$$

This is the optimal solution in the frequency domain. It can be written in the form of a filter applied to $\mathbf{G}$:

$$\hat{\mathbf{F}}(u, v) = \left[\frac{\mathbf{H}^*(u, v)}{|\mathbf{H}(u, v)|^2 + \lambda \mathbf{P}(u, v)} \right] \mathbf{G}(u, v)$$



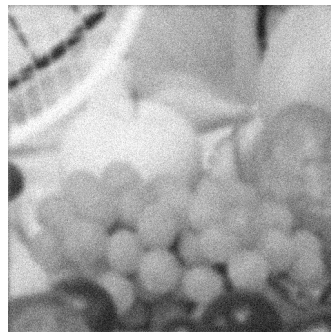
PSNR v/s λ (Reg. param.)



Best Restoration by Reg. deconv.

The best restoration using this approach had PSNR around 24.213dB which is a slight improvement standard wiener based restoration for same input noise level of 20dB (it's result can be seen in Part-D).

Additional Notes :



Here on the left side we keep the original input image to Q5 and on the right side we have kept Grayscale version of input image blurred with Gaussian PSF followed by an additive Gaussian Noise of magnitude 20dB.